

Guia de Estudio Completa - VIVA Assignment 2

Path Planning & Trajectory Generation - RMPC

Contents

1. VISION GENERAL DEL PROYECTO	3
Arquitectura de Archivos	3
Flujo de Datos Completo (main.py)	3
El Mapa de Obstaculos	3
Conceptos Clave del Lattice	3
2. ALGORITMO A* SOBRE LATTICE	5
2.1 Estructura del Grafo Lattice	5
2.2 Invalidacion de Obstaculos	5
2.3 Implementacion de A*	7
Tabla de conceptos clave	8
2.4 Reconstruccion del Camino	8
2.5 Heuristica: Distancia Euclidiana	9
2.6 Costo de Expansion	10
3. RRT - Rapidly-exploring Random Tree	11
3.1 Algoritmo Principal	11
3.2 Muestreo Aleatorio	11
3.3 Busqueda del Vecino Mas Cercano	12
3.4 Steer - Avanzar Hacia la Muestra	12
3.5 Deteccion de Colisiones	13
3.6 Goal Check y Reconstruccion de Path	13
4. PRM - Probabilistic Roadmap	15
Dos Fases: Construccion + Consulta	15
4.1 Fase 1: Construccion del Roadmap	15
4.2 Muestreo Libre de Obstaculos	16
4.3 K Vecinos Mas Cercanos con KD-Tree	16
4.4 Fase 2: Busqueda con Dijkstra	17
5. COMPARACION DE LOS 3 PLANIFICADORES	19
6. GENERACION DE TRAYECTORIA	20
6.1 Interpolacion del Path	20
6.2 Estaciones y Engranajes	20
6.3 Perfil de Velocidad Trapezoidal	21
Forward Pass - Aceleracion (lineas 124-131):	21
Backward Pass - Deceleracion (lineas 136-143):	21

Merge - Perfil Final (lineas 147-154):	22
6.4 Velocidad y Velocidad Angular	22
6.5 Aceleracion	23
6.6 Angulos Continuos	23
6.7 Normalize Angle	23
7. VISUALIZACION DE RESULTADOS	24
Trayectoria sobre el mapa	24
Trayectoria coloreada por velocidad	24
Perfil de orientacion	24
8. CODIGO PRE-CONSTRUIDO QUE DEBES ENTENDER	26
Clase Graph (<code>utils.py</code> lineas 6-31)	26
ObstaclesGrid (<code>utils.py</code> lineas 33-89)	26
<code>get_neighbor</code> (<code>lattice_planner.py</code> lineas 265-275)	26
9. BANCO DE PREGUNTAS RAPIDAS (FLASH CARDS)	27

1. VISION GENERAL DEL PROYECTO

El proyecto implementa **3 algoritmos de planificacion de caminos** y un **generador de trayectorias** para un robot movil en un entorno con obstaculos.

Arquitectura de Archivos

```
main.py          -- Orquestador: crea mapa, ejecuta planners, grafica
path_planner/
  utils.py       -- Estructura Graph + ObstaclesGrid (collision checker)
  lattice_planner.py -- Lattice graph + A*
  rrt_planner.py -- Rapidly-exploring Random Tree
  prm_planner.py -- Probabilistic Roadmap
trajectory_generator/
  traj_generation.py -- Resampling, perfil de velocidad, interpolacion
```

Flujo de Datos Completo (main.py)

1. Crear grid de lattice 10x10 (= 400 vertices con 4 orientaciones)
2. Definir obstaculos en mapa booleano 100x100
3. Invalidar aristas que cruzan obstaculos (peso = inf)
4. A* encuentra camino optimo en el lattice
5. RRT encuentra camino en espacio continuo
6. PRM construye roadmap y busca camino
7. Interpoliar el path del lattice (densificar)
8. Generar trayectoria con perfil de velocidad trapezoidal
9. Visualizar resultados

El Mapa de Obstaculos

Codigo que lo define (main.py lineas 37-48):

```
obs.map[25:35, 45:56] = True      # bloque superior-centro
obs.map[40:43, 48:76] = True      # barra horizontal
obs.map[67:89, 57:76] = True      # bloque inferior-derecha
obs.map[50:55, 60:89] = True      # barra inferior
obs.map[20:60, 5:35] = True       # bloque grande izquierda
```

- **Start:** (1, 8, 90) – fila 1, column 8, mirando “arriba” (90 grados)
- **Goal:** (8, 2, 270) – fila 8, column 2, mirando “abajo” (270 grados)

Conceptos Clave del Lattice

- **Vertice del lattice:** tupla (row, col, angle). Row y col son la posicion en el grid, angle es la orientacion del robot (0, 90, 180 o 270 grados).
- **Aristas rectas:** peso = 1 (una celda de distancia).
- **Aristas arco:** peso = pi (cuarto de circulo).
- **Obstaculos:** array booleano numpy 100x100. Una arista es invalida si cualquier punto muestreado a lo largo de ella cae en un obstaculo.

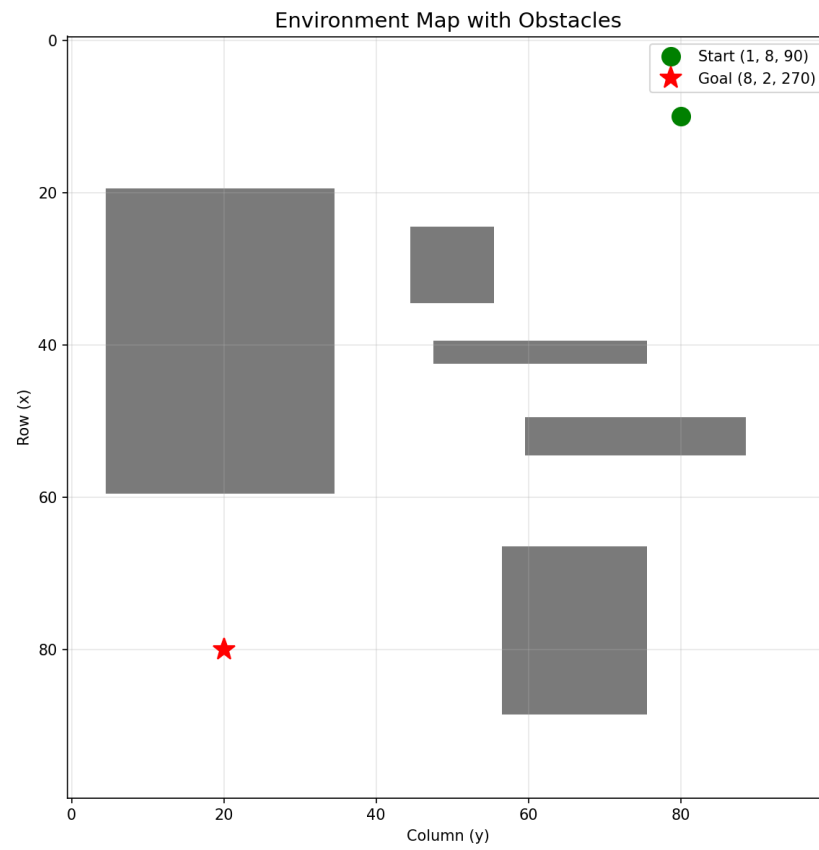


Figure 1: Mapa de obstaculos con start (1,8,90) y goal (8,2,270)

2. ALGORITMO A* SOBRE LATTICE

Archivo: `lattice_planner.py`

2.1 Estructura del Grafo Lattice

Cada celda del grid 10x10 tiene 4 orientaciones posibles = **400 vertices totales**.

Aristas rectas (peso = 1): El robot avanza una celda en la direccion de su heading.

- Angulo 0 $\rightarrow$ se mueve a la derecha (col+1)
- Angulo 90 $\rightarrow$ se mueve hacia arriba (row-1)
- Angulo 180 $\rightarrow$ se mueve a la izquierda (col-1)
- Angulo 270 $\rightarrow$ se mueve hacia abajo (row+1)

Aristas curvas/arco (peso = π): El robot gira 90 grados mientras avanza a una celda diagonal.

Codigo de generacion (`lattice_planner.py` lineas 66-183):

```
# Vertices: 4 angulos por cada celda (linea 75-79)
for row in range(n_rows):
    for col in range(n_cols):
        for angle in [0, 90, 180, 270]:
            v = (row, col, angle)
            self._graph.add_vertex(v)

# Arista recta - moverse hacia abajo (linea 101-103)
if (row + 1) < n_rows and angle == 270:
    v_bottom = (row + 1, col, 270)
    self._graph.set_edge(v, v_bottom, 1)           # peso = 1

# Arista curva - giro diagonal (linea 105-107)
if (col - 1) >= 0 and (row + 1) < n_rows and angle == 270:
    v_bottom_left = (row + 1, col - 1, 180)
    self._graph.set_edge(v, v_bottom_left, np.pi) # peso = pi
```

Pregunta Probable

"Por que el peso de un arco es pi y no sqrt(2)?"

El arco es un cuarto de circulo de radio = 1 celda. Su longitud es $(2\pi r)/4 = \pi r/2$. Con el radio normalizado, el codigo usa π como aproximacion del costo de recorrer esa curva. Es mayor que una linea recta (1) porque la curva es mas larga.

2.2 Invalidacion de Obstaculos

`update_obstacles()` (`lattice_planner.py` lineas 34-47):

```
def update_obstacles(self, obs):
    for edge_key, edge_val in self._graph._edge_dict.items():
        is_valid = obs.is_edge_valid(edge_key, edge_val,
                                     self.lattice_cell_size, self.arc_primitives)
        if not is_valid:
```

```
self._graph._edge_dict[edge_key] = np.inf # arista bloqueada  
self._graph.set_adjacency_matrix()
```

Se muestrea cada arista (linea recta o arco) y si **cualquier punto** cae dentro de un obstaculo, el peso se pone en **inf**. A* nunca elegira esa arista.

2.3 Implementacion de A*

Lineas 209-238 de lattice_planner.py

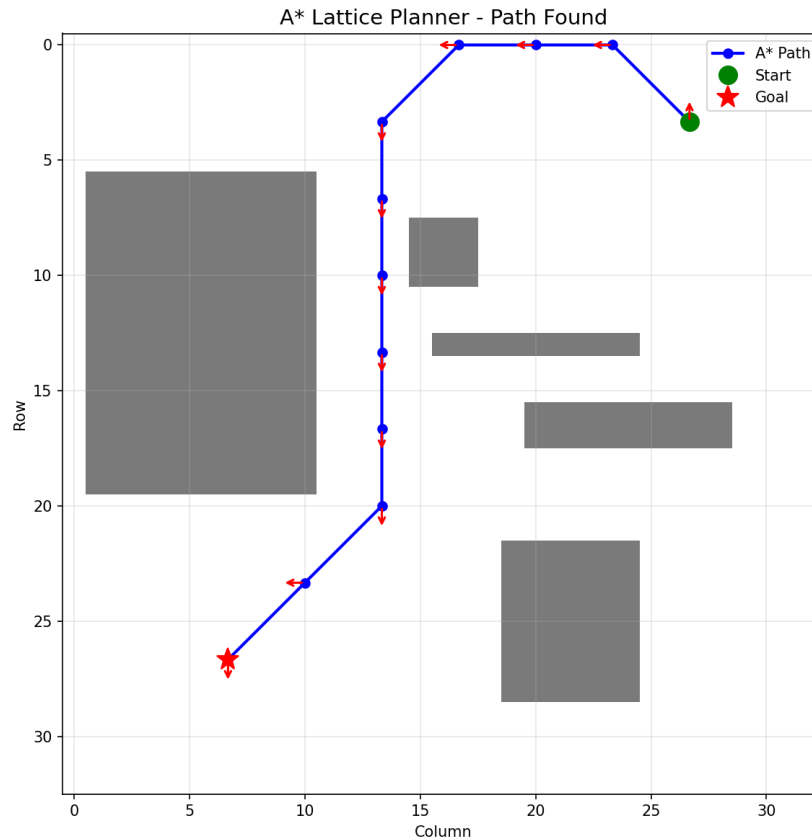


Figure 2: Camino encontrado por A* sobre el lattice

```
distances[s] = 0                                # g(start) = 0
costs[s] = self.calH(s, g)                       # f(start) = 0 + h(start)

while not open_set.empty():
    cost, curr = open_set.get()                   # pop nodo con menor f

    if curr in closed_set:                        # lazy deletion: saltar duplicados
        continue
    closed_set.add(curr)                          # marcar como expandido

    if curr == g:                                # goal check DESPUES de popping
        path = self.traverse_path(s, g, parent_node)
        return path

    neighbors = self.get_neighbor(curr, graph_vert_list, adjacency_matrix)
    for n in neighbors:
        if n in closed_set:
            continue

        g_cost = distances[curr] + self.cal_expand_cost(curr, n, edge_dict)
```

```

    if n not in distances or g_cost < distances[n]:
        distances[n] = g_cost
        f = g_cost + self.calH(n, g)      # f = g + h
        costs[n] = f
        parent_node[n] = curr
        open_set.put((f, n))              # push con prioridad f

print("no path found")
return None

```

Tabla de conceptos clave

Concepto	Que es	En tu codigo
g-cost	Costo real desde start hasta nodo	<code>distances[n]</code>
h-cost	Estimacion desde nodo hasta goal	<code>calH(n, g)</code>
f-cost	g + h, costo total estimado	<code>f = g_cost + self.calH(n, g)</code>
Open set	Nodos descubiertos no expandidos	<code>PriorityQueue (min-heap)</code>
Closed set	Nodos ya expandidos	<code>set()</code>
Lazy deletion	Push duplicados, skip al pop	<code>if curr in closed_set: continue</code>

Pregunta Probable

"Por que revisas el goal DESPUES de sacarlo de la cola y no cuando lo agregas?"

Si lo reviso al agregar, podria encontrar el goal por un camino suboptimo primero. Al revisarlo despues de popping, el PriorityQueue garantiza que la primera vez que saco el goal, su f-cost es el minimo. Con una heuristica admisible, esto garantiza optimalidad.

Pregunta Probable

"Que es lazy deletion y por que la usas?"

El PriorityQueue de Python no tiene operacion `decrease-key`. Cuando encuentro un mejor camino a un nodo, no puedo actualizar su prioridad en la cola. En su lugar, push una nueva entrada con la nueva prioridad. Cuando pop una entrada vieja (el nodo ya esta en `closed_set`), simplemente la ignoro. Es correcto y $O(E \log V)$ en practica.

2.4 Reconstruccion del Camino

Lineas 255-262 de `lattice_planner.py`:

```

path = []
curr = g
while curr != s:
    path.append(curr)
    curr = parent_node[curr]
path.append(s)
path.reverse()
return path

```

Backtracking clasico: desde goal, sigo los parent pointers hasta start, luego invierto.

Pregunta Probable

"Por que no construyes el camino hacia adelante desde start?"

Durante la busqueda, solo registro el padre de cada nodo (`parent_node[child] = parent`), no sus hijos. Solo puedo trazar de hijo a padre, por eso voy de goal a start y luego invierto.

2.5 Heuristica: Distancia Euclidiana

Lineas 307-309 de `lattice_planner.py`:

```

dx = v1[0] - v2[0]
dy = v1[1] - v2[1]
return np.sqrt(dx**2 + dy**2)

```

Distancia euclidiana entre (row, col) del nodo actual y el goal. Ignora el angulo.

Pregunta Probable

"Es tu heuristica admisible? Demuestralo."

Si. La distancia euclidiana es la linea recta entre dos puntos, que es siempre $\leq$ cualquier camino real (que debe recorrer aristas de peso 1 o π). Nunca sobreestima, por lo tanto es admisible.

Pregunta Probable

"Es consistente (monotona)?"

Si. Para cualquier nodo n y vecino n' : $h(n) \leq cost(n, n') + h(n')$. Esto se cumple por la desigualdad triangular de la distancia euclidiana. Consistencia implica admisibilidad, y garantiza que A^* no necesita reabrir nodos.

Pregunta Probable

"Por que ignoras el angulo en la heuristica?"

La heuristica solo necesita ser una cota inferior. Incluir diferencias de angulo la haria mas informada pero tambien mas compleja. La euclidiana en (row,col) ya es admisible y funciona bien.

Pregunta Probable

"Podrias usar Manhattan como heuristica?"

Si, pero seria menos informada que euclidiana porque el lattice permite movimientos diagonales via arcos. Manhattan sobreestimaria para caminos diagonales. Euclidiana es mas tight (mas cercana al costo real).

2.6 Costo de Expansion

Linea 291 de `lattice_planner.py`:

```
return edge_dict[(v1, v2)]
```

Busca el peso de la arista en el diccionario. Rectas = 1, arcos = π , bloqueadas = ∞ .

3. RRT - Rapidly-exploring Random Tree

Archivo: rrt_planner.py

Parametros: max_iter=500, step_size=5, mapa 100x100

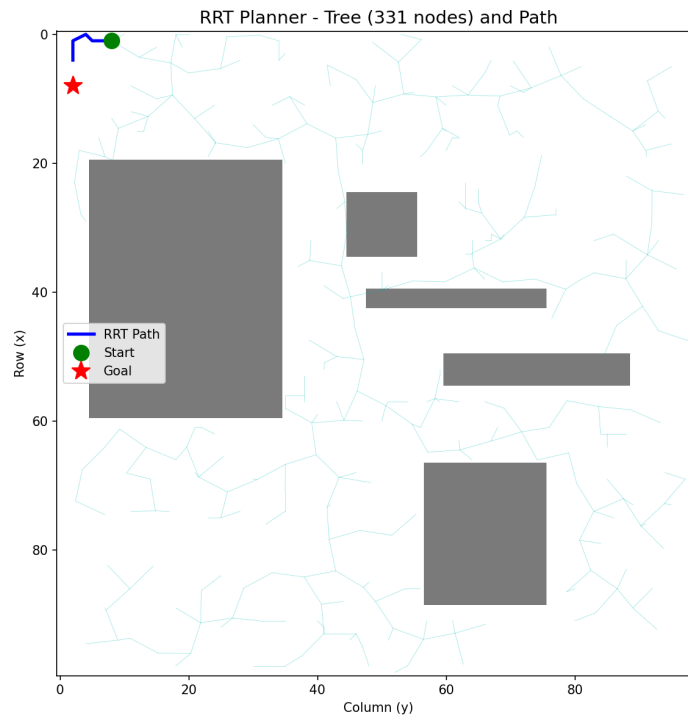


Figure 3: Arbol RRT y camino encontrado

3.1 Algoritmo Principal

Lineas 40-59 de rrt_planner.py:

```
def plan(self):
    for i in range(self.max_iter):
        rand_node = self.sample_random_point()           # 1. Muestra aleatoria
        nearest_node = self.find_nearest_node(rand_node) # 2. Nodo mas cercano
        new_node = self.steer(nearest_node, rand_node)   # 3. Avanzar

        if new_node and not self.is_colliding(new_node, nearest_node):
            self.tree.append(new_node)                   # 4. Agregar al arbol
            if self.reached_goal(new_node):              # 5. Llegamos?
                return self.construct_path(new_node)

    print("Path not found.")
    return None
```

3.2 Muestreo Aleatorio

Lineas 68-70:

```
rx = np.random.randint(0, self.map_size[0])
ry = np.random.randint(0, self.map_size[1])
return Node(rx, ry)
```

Muestreo uniforme en el mapa. Usa enteros porque el mapa de obstaculos es un grid discreto.

Pregunta Probable

"Por que no agregas goal biasing?"

Goal biasing (muestrear el goal un 5-10% del tiempo) aceleraria la convergencia. Lo mantuve simple porque con 500 iteraciones y step_size=5 en un mapa 100x100, converge bien. No era requerido por el assignment.

3.3 Busqueda del Vecino Mas Cercano

Lineas 82-89:

```
best = None
best_d = float('inf')
for n in self.tree:
    d = np.sqrt((n.x - rand_node.x)**2 + (n.y - rand_node.y)**2)
    if d < best_d:
        best_d = d
        best = n
return best
```

Busqueda lineal $O(n)$. Recorre todos los nodos del arbol.

Pregunta Probable

"Esto es $O(n)$. Como lo mejorarias?"

Usando un **KD-Tree** (como `scipy.spatial.KDTree`) para queries $O(\log n)$. De hecho, en mi PRM planner si use KDTree. Para RRT con max 500 iteraciones, la busqueda lineal es suficiente.

3.4 Steer - Avanzar Hacia la Muestra

Lineas 102-109:

```
dx = rand_node.x - nearest_node.x
dy = rand_node.y - nearest_node.y
d = np.sqrt(dx**2 + dy**2)
if d <= self.step_size:
    return Node(rand_node.x, rand_node.y, nearest_node)
nx = nearest_node.x + dx / d * self.step_size # vector unitario * step_size
ny = nearest_node.y + dy / d * self.step_size
return Node(nx, ny, nearest_node)
```

Si la muestra esta dentro de step_size, la usa directamente. Si no, avanza exactamente step_size unidades en esa direccion. dx/d y dy/d forman el vector unitario de direccion.

Pregunta Probable

"Por que limitar el step size?"

Sin limite, el arbol podria saltar sobre obstaculos. El step_size asegura crecimiento incremental y que el collision check entre los dos nodos sea confiable sobre distancias cortas. Tambien controla la densidad del arbol.

3.5 Deteccion de Colisiones

Lineas 122-132:

```
dist = np.sqrt((new_node.x - nearest_node.x)**2 +
               (new_node.y - nearest_node.y)**2)
n_steps = max(int(dist), 1)
for i in range(n_steps + 1):
    t = i / n_steps
    px = int(nearest_node.x + t * (new_node.x - nearest_node.x))
    py = int(nearest_node.y + t * (new_node.y - nearest_node.y))
    if px < 0 or px >= self.map_size[0] or \
       py < 0 or py >= self.map_size[1]:
        return True
    if self.obstacles.map[px, py]:
        return True
return False
```

Muestrea puntos a lo largo del segmento de linea (~1 punto por unidad de distancia). Si cualquier punto esta en obstaculo o fuera del mapa, hay colision.

Pregunta Probable

"Podria tu collision check fallar (miss un obstaculo)?"

Con n_steps = int(dist) muestreo ~1 punto por unidad. Como los obstaculos ocupan celdas de 1x1, es generalmente suficiente. Para un check perfecto, usaria el algoritmo de Bresenham para enumerar todas las celdas que la linea atraviesa. Pero para step_size=5, la densidad de muestreo es adecuada.

3.6 Goal Check y Reconstruccion de Path

reached_goal (lineas 144-145):

```
d = np.sqrt((new_node.x - self.goal.x)**2 +
            (new_node.y - self.goal.y)**2)
return d <= self.step_size
```

construct_path (lineas 157-163):

```
path = []
node = end_node
while node is not None:           # root tiene parent = None
    path.append(node)
    node = node.parent
```

```
path.append((node.x, node.y))
node = node.parent
path.reverse()
return path
```

Pregunta Probable

"Es RRT optimo?"

No. RRT es **probabilisticamente completo** (si existe un camino, lo encontrara con suficientes iteraciones), pero NO garantiza el camino mas corto. Para optimalidad se necesita **RRT***, que re-conecta el arbol para reducir costos.

Pregunta Probable

"Que es completitud probabilistica?"

Conforme el numero de iteraciones tiende a infinito, la probabilidad de encontrar un camino (si existe) tiende a 1. No es determinista: una ejecucion individual puede fallar, pero con suficiente tiempo, tendra exito.

Pregunta Probable

"Como afecta step_size al rendimiento de RRT?"

Muy pequeno: expansion lenta, muchas iteraciones. Muy grande: podria saltar sobre pasajes estrechos, collision checks menos confiables. step_size=5 en mapa 100x100 es un buen balance.

4. PRM - Probabilistic Roadmap

Archivo: prm_planner.py

Parametros: num_samples=200, k_neighbors=10, step_size=5

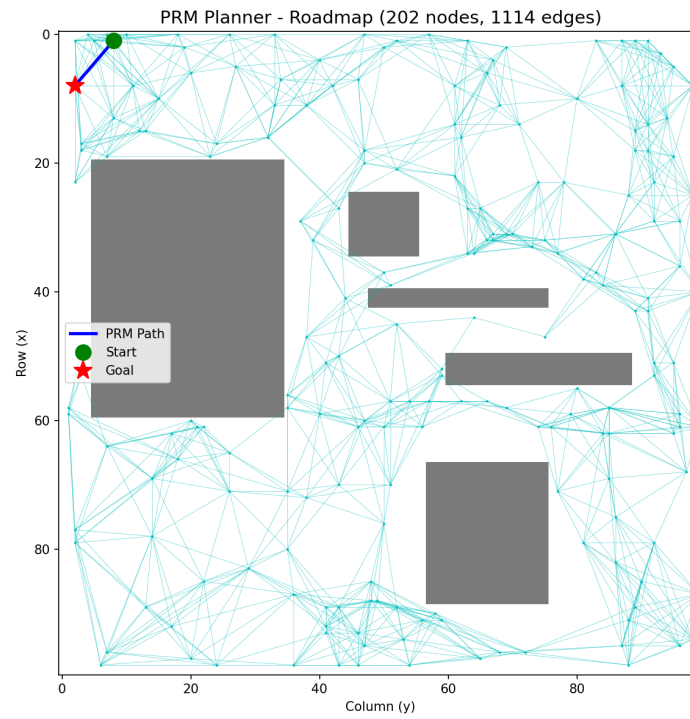


Figure 4: Roadmap PRM con 202 nodos y 1114 aristas

Dos Fases: Construcción + Consulta

4.1 Fase 1: Construcción del Roadmap

Lineas 49-69 de prm_planner.py:

```
# Agregar start y goal primero
self.roadmap.append(self.start)
self.roadmap.append(self.goal)

# Muestrear puntos libres de obstaculos
for _ in range(self.num_samples):
    p = self.sample_free_point()
    if p is not None:
        self.roadmap.append(p)

# Conectar cada nodo a sus k vecinos mas cercanos
for node in self.roadmap:
    nearest = self.find_k_nearest(node, self.k_neighbors)
    for nb in nearest:
        if not self.is_colliding(node, nb):
            # Aristas bidireccionales (grafo no dirigido)
            if node not in self.edges:
```

```

        self.edges[node] = []
    if nb not in self.edges:
        self.edges[nb] = []
    if nb not in self.edges[node]:
        self.edges[node].append(nb)
    if node not in self.edges[nb]:
        self.edges[nb].append(node)

```

Pregunta Probable

"Por que agregas start y goal primero?"

Si no estan en el roadmap, no se puede encontrar un camino hacia/desde ellos. Deben participar en el proceso de conexion con vecinos.

4.2 Muestreo Libre de Obstaculos

Lineas 78-83:

```

for _ in range(100):    # hasta 100 intentos (rejection sampling)
    x = np.random.randint(0, self.map_size[0])
    y = np.random.randint(0, self.map_size[1])
    if not self.obstacles.map[x, y]:
        return Node(x, y)
return None

```

4.3 K Vecinos Mas Cercanos con KD-Tree

Lineas 96-107:

```

pts = np.array([[n.x, n.y] for n in self.roadmap])
tree = KDTree(pts)
k_actual = min(k + 1, len(self.roadmap)) # k+1 porque incluye al propio nodo
_, idxs = tree.query([node.x, node.y], k=k_actual)

result = []
for i in idxs:
    nb = self.roadmap[i]
    if nb.x == node.x and nb.y == node.y: # excluir el propio nodo
        continue
    result.append(nb)
return result[:k]

```

Pregunta Probable

"Reconstruyes el KDTree en cada llamada. No es ineficiente?"

Si, $O(n \log n)$ por llamada $\times n$ llamadas = $O(n^2 \log n)$ total. Idealmente construiria el tree una sola vez. Con 200 nodos es negligible.

Pregunta Probable

"Por que KDTree en PRM pero busqueda lineal en RRT?"

PRM conecta TODOS los nodos a sus k vecinos, llamando find_k_nearest 200+ veces. KDTree hace cada query $O(\log n)$ en vez de $O(n)$. Para RRT, find_nearest se llama max 500 veces en un arbol creciente, y la busqueda lineal simple era suficiente.

4.4 Fase 2: Busqueda con Dijkstra

Lineas 139-189 de prm_planner.py:

```
from queue import PriorityQueue
dist = {}; prev = {}; visited = set()
pq = PriorityQueue()

sk = (self.start.x, self.start.y)  # tuplas como claves
gk = (self.goal.x, self.goal.y)
dist[sk] = 0
pq.put((0, sk))

lookup = {}  # tupla -> nodo
for n in self.roadmap:
    lookup[(n.x, n.y)] = n

while not pq.empty():
    c, curr = pq.get()
    if curr in visited: continue
    visited.add(curr)

    if curr == gk:  # encontramos el goal
        path = []; k = gk
        while k in prev:
            path.append(k); k = prev[k]
        path.append(sk); path.reverse()
        return path

    node = lookup.get(curr)
    if node is None or node not in self.edges: continue

    for nb in self.edges[node]:
        nbk = (nb.x, nb.y)
        if nbk in visited: continue
        w = np.sqrt((node.x - nb.x)**2 + (node.y - nb.y)**2)
        new_c = dist[curr] + w
        if nbk not in dist or new_c < dist[nbk]:
            dist[nbk] = new_c
            prev[nbk] = curr
            pq.put((new_c, nbk))
```

```
print("PRM: no path found")  
return None
```

Es **Dijkstra** (A^* con $h=0$). Los pesos son las distancias euclidianas entre nodos conectados.

Pregunta Probable

"Por que Dijkstra en lugar de A^* ?"

Ambos funcionan. Dijkstra es mas simple (no necesita heuristica). Con ~ 200 nodos, la diferencia de rendimiento es despreciable.

Pregunta Probable

"Por que tuplas como claves de diccionario en vez de objetos Node?"

Los objetos Node no son hashables por contenido (se hashean por identidad). Dos objetos Node con las mismas coordenadas serian claves diferentes. Las tuplas (x,y) se comparan por valor, lo cual es correcto.

5. COMPARACION DE LOS 3 PLANIFICADORES

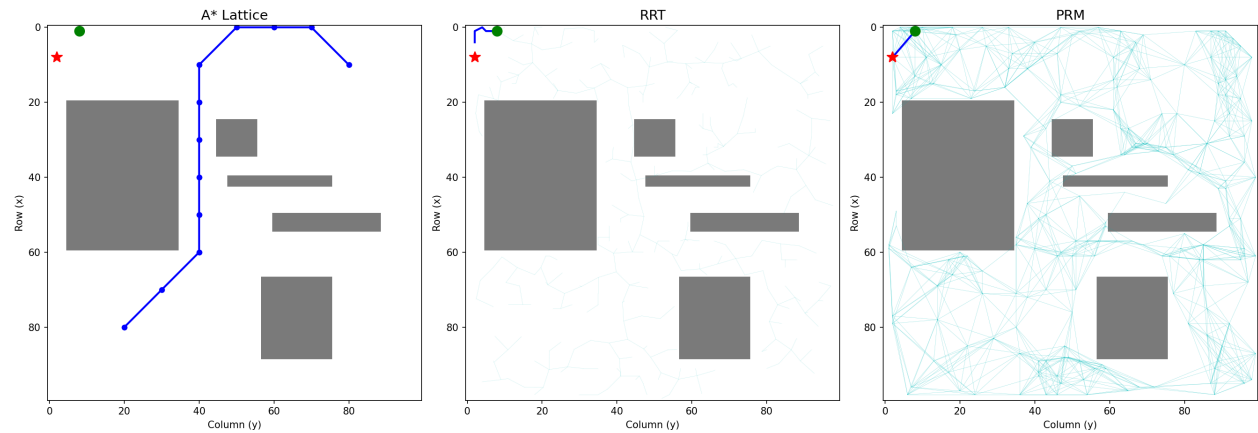


Figure 5: Comparacion visual: A* Lattice, RRT y PRM

Propiedad	A* Lattice	RRT	PRM
Tipo de espacio	Discreto (lattice)	Continuo	Continuo
Optimalidad	Si (con h admisible)	No	No (aprox.)
Compleitud	Completo	Prob. completo	Prob. completo
Multi-query	Si (mismo grafo)	No (arbol nuevo)	Si (roadmap reusable)
Orientacion	Si (4 headings)	No (solo x,y)	No (solo x,y)

Pregunta Probable

"Cual planificador es mejor?"

Depende del contexto: **A*** cuando necesitas optimalidad y el robot tiene restricciones de orientacion. **RRT** para single-query en espacios de alta dimension. **PRM** cuando necesitas resolver muchas queries en el mismo entorno.

6. GENERACION DE TRAYECTORIA

Archivo: traj_generation.py

La trayectoria convierte un **path discreto** (waypoints) en un **perfil continuo** con posicion, velocidad, aceleracion y velocidad angular parametrizados por tiempo.

6.1 Interpolacion del Path

Lineas 164-192 de traj_generation.py. Densifica el path del lattice:

```
# Segmentos rectos (misma orientacion): lineas 182-186
if path[i][2] == path[i+1][2]:
    for j in range(lattice_cell_size):
        row = v1[0]*lattice_cell_size + j*dir_row/lattice_cell_size
        col = v1[1]*lattice_cell_size + j*dir_col/lattice_cell_size
        sampled_path.append((row/cell_size, col/cell_size, v1[2]))

# Arcos (diferente orientacion): lineas 187-191
else:
    arc = graph.arc_primitives[(v1[2], v2[2])]
    arc = np.array(v1[:2]).reshape((2,1)) * lattice_cell_size + arc
    for j in range(arc.shape[1]):
        sampled_path.append((arc[0,j]/cell_size, arc[1,j]/cell_size, ...))
```

6.2 Estaciones y Engranajes

Lineas 30-47:

```
for i in range(1, len(path)):
    d = self.distance(path[i], path[i-1])
    stations[i] = stations[i-1] + d      # distancia acumulada

    # Determinar si avanza o retrocede
    d_row = path[i][0] - path[i-1][0]
    d_col = path[i][1] - path[i-1][1]
    avg_th = (path[i-1][2] + path[i][2]) / 2.0
    h_row = -math.sin(math.radians(avg_th)) # heading en coords grid
    h_col = math.cos(math.radians(avg_th))
    dot = d_row * h_row + d_col * h_col     # producto punto
    if dot >= 0: gears[i] = 1      # forward
    else:      gears[i] = -1     # reverse
```

- **Station:** distancia acumulada a lo largo del path.
- **Gear:** producto punto entre desplazamiento y heading. Positivo = forward, negativo = reverse.

Pregunta Probable

"Por que usas -sin para el componente de fila?"

En coordenadas de grid, la fila aumenta hacia ABAJO. En matematicas, y aumenta hacia arriba. Heading 90 grados (hacia "arriba") significa que la fila DISMINUYE. El heading en direccion de fila es $-\sin(\theta)$.

6.3 Perfil de Velocidad Trapezoidal

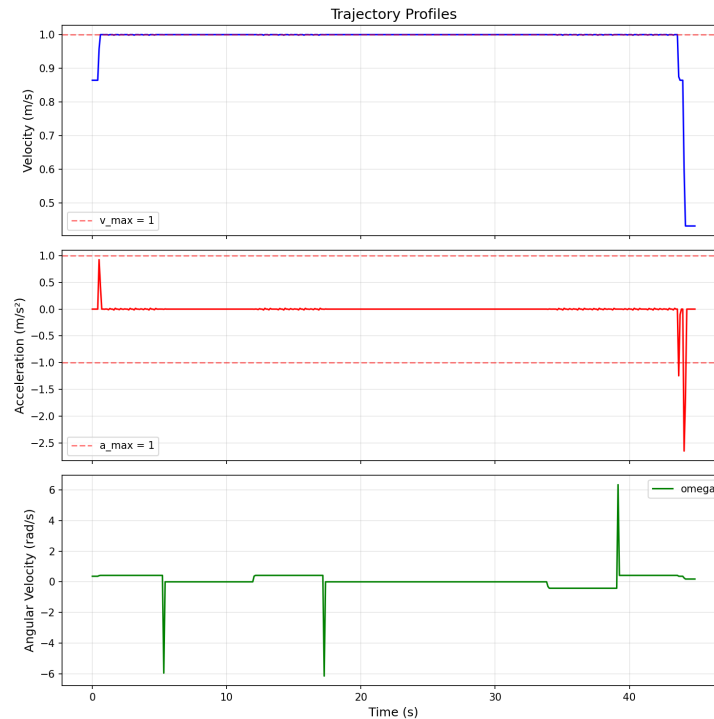


Figure 6: Perfiles de velocidad, aceleracion y velocidad angular

Usa la ecuacion cinematica: $v_f^2 = v_i^2 + 2 \cdot a \cdot \Delta s$

Forward Pass - Aceleracion (lineas 124-131):

```
fwd = [0.0] * len(stations)
for i in range(1, len(stations)):
    ds = stations[i] - stations[i-1]
    if ds < 1e-9:
        fwd[i] = fwd[i-1]
    else:
        v2 = fwd[i-1]**2 + 2 * max_accel * ds
        fwd[i] = min(math.sqrt(max(v2, 0)), max_velocity)
```

Maxima velocidad alcanzable en cada punto **si el robot acelera desde reposo.**

Backward Pass - Deceleracion (lineas 136-143):

```
bwd = [0.0] * len(stations)
for i in range(len(stations) - 2, -1, -1):
    ds = stations[i+1] - stations[i]
```

```

if ds < 1e-9:
    bwd[i] = bwd[i+1]
else:
    v2 = bwd[i+1]**2 + 2 * max_accel * ds
    bwd[i] = min(math.sqrt(max(v2, 0)), max_velocity)

```

Misma ecuacion pero **hacia atras desde el final** (donde debe detenerse).

Merge - Perfil Final (lineas 147-154):

```

for i in range(len(stations)):
    profile[i] = min(fwd[i], bwd[i])    # minimo de ambas envolventes

```

Esto crea el perfil trapezoidal clasico: acelerar → crucero → decelerar.

Pregunta Probable

"Explica el perfil de velocidad trapezoidal."

El robot arranca en reposo, acelera a `max_acceleration` hasta alcanzar `max_velocity` (o hasta que necesite frenar), cruza a `max_velocity`, y decelera para detenerse en el goal. Es el perfil **optimo en tiempo** bajo restricciones de aceleracion constante. El forward pass computa la envolvente "acelerando desde reposo", el backward pass computa "decelerando hasta reposo", y `min()` da el perfil factible.

Pregunta Probable

"De donde viene la ecuacion $v_f^2 = v_i^2 + 2as$?"

De las ecuaciones cinematicas. Combinando $v_f = v_i + at$ y $s = v_it + \frac{1}{2}at^2$, eliminando el tiempo t , se obtiene $v_f^2 = v_i^2 + 2as$.

Pregunta Probable

"Que pasa si el camino es muy corto para alcanzar `max_velocity`?"

Las envolventes forward y backward se cruzan antes de llegar a `max_velocity`, creando un perfil **triangular** (acelera e inmediatamente decelera, sin fase de crucero). El `min()` lo maneja automaticamente.

6.4 Velocidad y Velocidad Angular

Lineas 86-95:

```

for i in range(1, nfe):
    dx = result.states[i].x - result.states[i-1].x
    dy = result.states[i].y - result.states[i-1].y
    result.states[i].v = math.sqrt(dx**2 + dy**2) / dt    # v = |despl| / dt

```

```

    dth = result.states[i].theta - result.states[i-1].theta
    result.states[i].omega = dth / dt                # omega = dtheta/dt
if nfe > 1:
    result.states[0].v = result.states[1].v          # condicion de frontera
    result.states[0].omega = result.states[1].omega

```

Diferencias finitas hacia atras. Primer elemento se copia del segundo (no hay indice -1).

6.5 Aceleracion

Lineas 99-102:

```

for i in range(1, nfe):
    result.states[i].a = (result.states[i].v - result.states[i-1].v) / dt
if nfe > 1:
    result.states[0].a = result.states[1].a

```

6.6 Angulos Continuos

Lineas 258-269 - to_continuous_angle():

```

rads = []
for a in angles:
    rads.append(math.radians(a))    # grados -> radianes
out = [rads[0]]
for i in range(1, len(rads)):
    diff = rads[i] - rads[i-1]
    diff = self.normalize_angle(diff)    # normalizar a [-pi, pi]
    out.append(out[-1] + diff)          # acumular suavemente
return out

```

Pregunta Probable

"Que hace to_continuous_angle y por que es necesaria?"

Sin ella, interpolar entre heading 350 deg y 10 deg iria por 180, 90, 0 en vez de cruzar suavemente por 360/0 (+20 grados). La representacion continua "desenvuelve" los angulos para que la interpolacion no tenga saltos. Esto evita que el robot gire salvajemente.

6.7 Normalize Angle

Lineas 229-233:

```

while angle > math.pi:
    angle -= 2 * math.pi
while angle < -math.pi:
    angle += 2 * math.pi
return angle

```

Envuelve cualquier angulo al rango $[-\pi, \pi]$.

7. VISUALIZACION DE RESULTADOS

Trayectoria sobre el mapa

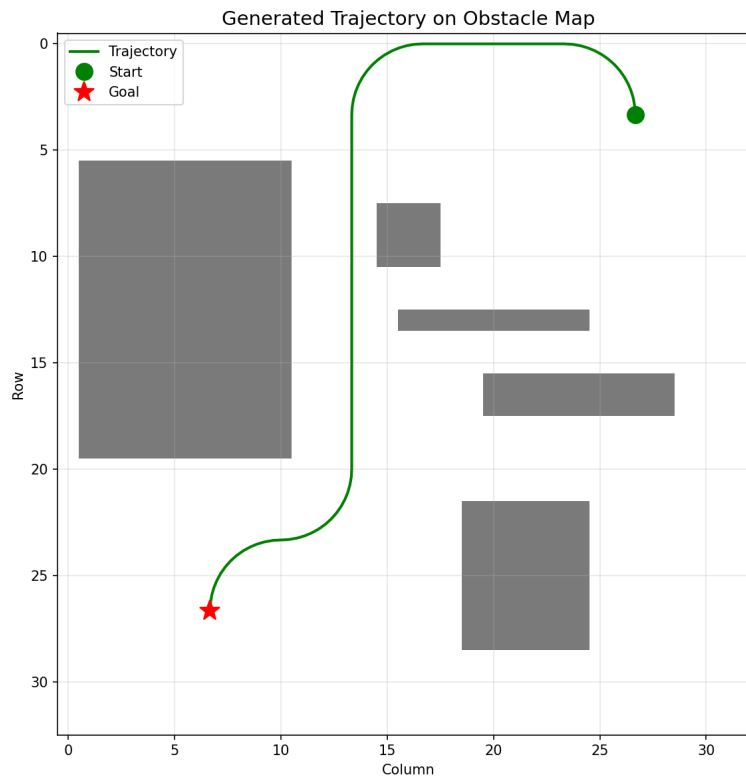


Figure 7: Trayectoria generada sobre el mapa de obstaculos

La trayectoria (línea verde) sigue el path del A* interpolado, evitando obstaculos con curvas suaves.

Trayectoria coloreada por velocidad

Colores oscuros = baja velocidad (inicio/fin, curvas). Colores claros/amarillos = alta velocidad (segmentos rectos, crucero).

Perfil de orientacion

Inicia en 90 grados, gira por las curvas, y termina cerca de 270 grados (heading del goal).

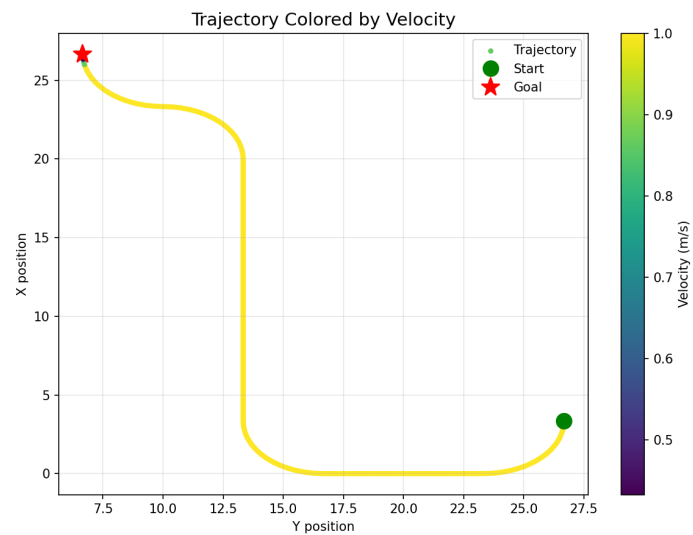


Figure 8: Trayectoria coloreada por velocidad: oscuro = lento, claro = rapido

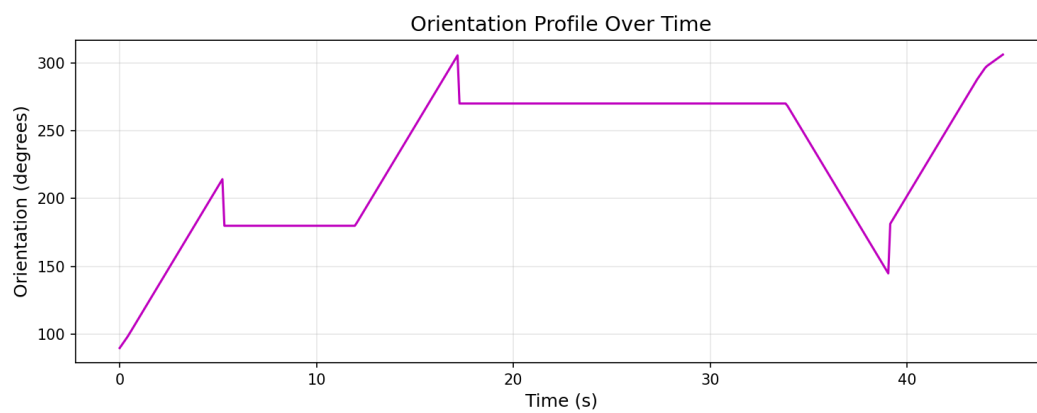


Figure 9: Orientacion del robot a lo largo del tiempo

8. CODIGO PRE-CONSTRUIDO QUE DEBES ENTENDER

Clase Graph (utils.py lineas 6-31)

```
class Graph:
    _vert_list = []           # Lista de vertices (row, col, angle)
    _edge_dict = {}          # (v1, v2) -> peso
    _adjacency_matrix        # Matriz NxN de pesos
```

Nota: La adjacency matrix tiene convencion transpuesta: `_adjacency_matrix[j, i] = edge_dict[(u, v)]`.

ObstaclesGrid (utils.py lineas 33-89)

- `map`: Boolean numpy array (100x100). True = obstaculo.
- `is_edge_valid()`: Muestra puntos a lo largo de una arista y verifica colisiones.
- `is_point_valid()`: Verifica que un punto no esta en obstaculo ni fuera del mapa.

`get_neighbor` (lattice_planner.py lineas 265-275)

```
def get_neighbor(self, u, graph_vert_list, adjacency_matrix):
    row = graph_vert_list.index(u)
    is_adj = (adjacency_matrix[row, :] < np.inf) & \
             (adjacency_matrix[row, :] > 0)
    adj_list = []
    for i, v in enumerate(graph_vert_list):
        if is_adj[i]:
            adj_list.append(v)
    return adj_list
```

9. BANCO DE PREGUNTAS RAPIDAS (FLASH CARDS)

#	Pregunta	Respuesta
1	Que estructura de datos usa A*?	Priority Queue (min-heap), set, dicts
2	Diferencia A* vs Dijkstra?	A* usa heuristica $h(n)$. $f = g + h$
3	Que es "admisible"?	h nunca sobreestima el costo real
4	Que es "consistente"?	$h(n) \leq \text{cost}(n, n') + h(n')$
5	Cuantos vertices en tu lattice?	$10 \times 10 \times 4 = 400$
6	Peso arista recta?	1
7	Peso arista arco?	π
8	Como se bloquean obstaculos?	Peso = ∞
9	Es RRT optimo?	No. Prob. completo, no optimo
10	Que es PRM?	Sample espacio libre, conectar, buscar
11	Fases de PRM?	Construccion + Consulta
12	Cuando PRM > RRT?	Multiples queries, mismo entorno
13	Perfil trapezoidal?	Acelerar $\rightarrow$ crucero $\rightarrow$ decelerar
14	$v_f^2 = v_i^2 + 2as$ viene de?	Eliminar t de $v = v_0 + at$ y $s = v_0t + \frac{1}{2}at^2$
15	Por que forward Y backward?	$\min(\text{acelerando}, \text{decelerando}) = \text{factible}$
16	Velocidad angular?	$\omega = d\theta/dt$
17	Por que deg a rad?	Formulas fisicas usan radianes
18	normalize_angle?	Envuelve a $[-\pi, \pi]$
19	to_continuous_angle?	Desenvuelve para evitar saltos en 0/360
20	Que es el gear?	Forward (1) o reverse (-1), por dot product
21	Que es una station?	Distancia acumulada a lo largo del path
22	Lattice cell size?	10 unidades
23	Que pasa si no hay camino A*?	Cola se vacia, retorna None
24	Complejidad A*?	$O(V \log V + E \log V)$
25	Que es lazy deletion?	Push duplicados, skip stale al pop
26	Goal check: al push o al pop?	Al pop (garantiza optimalidad)
27	Si h no es admisible?	A* puede dar camino suboptimo
28	Que es completitud probabilistica?	$P(\text{encontrar path}) \rightarrow 1$ con $\text{iter} \rightarrow \infty$
29	Diferencia collision check RRT vs PRM?	Misma idea, diferente densidad de muestreo
30	Adjacency matrix almacena?	Pesos. 0=sin arista, +val=peso, ∞ =bloqueada